



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ ДЕПАРТАМЕНТ
МАТЕМАТИЧЕСКОГО И КОМПЬЮТЕРНОГО МОДЕЛИРОВАНИЯ

Курсовой проект на тему

Методы продолжения по параметру для решения систем нелинейных уравнений

Выполнил: Студент группы Б9122-01.03.02 МКТ
Лобанов Илья Вячеславович

Проверил:

г. Владивосток
2025г.

Содержание

Введение	2
1 Метод продолжения по параметру	3
1.1 Краткое описание	3
1.2 Замена параметра	6
1.3 Наилучшая параметризация	8
2 Численные схемы	14
2.1 Конечно-разностные схемы	15
2.1.1 Простой метод Эйлера	15
2.1.2 Модифицированный метод Эйлера	16
2.1.3 Метод Рунге-Кутты четвертого порядка точности	17
2.2 Возможные эвристики	18
2.2.1 Сохранение ориентации кривой	18
2.2.2 Динамический размер шага	19
Заключение	20
Список литературы	21
A Программная реализация	22
B Примеры и тесты	30

Введение

Одной из самых главных задач математики и ее приложений является решение уравнений различной природы: алгебраических или трансцендентных, дифференциальных или интегральных, смешанных и прочих. Эта задача настолько важна, что отдельному классу уравнений посвящен целый раздел математики — линейная алгебра.

К сожалению, далеко не каждый класс уравнений поддается исчерпывающему аналитическому описанию. Даже для многочленов выше четвертой степени нахождение корней представляется нетривиальной задачей.

При этом часто удается свести исходную задачу к более простой. И, наоборот, задачу, решение которой уже известно или вычисляется относительно просто, «продеформировать» особым образом (об этом далее) в исходную. Тут и являет себя во всей красе метод продолжения по параметру.

Первое использование идеи продолжения в вычислительных целях принадлежит М. Лаэю [6] (1934 г.). Он пытался разрешить проблему подбора начального приближения в методе Ньютона — итерационного способа решения нелинейных уравнений. Его идея использовать на каждом новом шаге продвижения по параметру результаты, полученные на предыдущих шагах, лежит в основе рассматриваемого метода. Такой подход называют дискретным продолжением по параметру.

Немногим позже уже советский математик Д.Ф. Давиденко [5] (1953 г.) применил к процессу продолжения математический аппарат дифференциальных уравнений, тем самым получив непрерывный метод продолжения по параметру.

Непрерывный подход более удобен в аналитическом исследовании метода, в то время как дискретный — в вычислительных целях.

В этой работе будет рассмотрен упомянутый метод: дано подробное описание, рассмотрены возможные проблемы и способы их решения. В конце будет сделана компьютерная реализация метода на языке программирования Python и проведены соответствующие тесты.

1 Метод продолжения по параметру

1.1 Краткое описание

Рассмотрим систему из n уравнений с n неизвестными, которую в евклидовом пространстве $\mathbb{R}^n$ можно представить в операторном виде

$$F(x) = 0, \quad (1.1)$$

где $F : \Omega \rightarrow \mathbb{R}^n$, а Ω — область в $\mathbb{R}^n$.

Решать систему (1.1) в общем случае проблематично, поэтому предлагается ввести такую функцию $G : \Omega \rightarrow \mathbb{R}^n$, которая, во-первых, гладко «деформируется» в F , а, во-вторых, уравнение $G(x) = 0$ решается относительно просто.

Определение 1.1.1. Если для непрерывных отображений $F, G : \Omega \rightarrow \mathbb{R}^n$, где Ω область в $\mathbb{R}^n$, существует непрерывное отображение $H(x, \mu) : \Omega \times [0, 1] \rightarrow \mathbb{R}^n$ для которого верны следующие равенства для любых $x \in \Omega$:

1. $H(x, 1) = F(x)$,
2. $H(x, 0) = G(x)$.

Иными словами $H(x, \mu)$ деформирует $G(x)$ в $F(x)$ по параметру μ в классе $C^0(\Omega)$ (непрерывных на области Ω отображений). Тогда отображение $H(x, \mu)$ называют гомотопией, а F и G — гомотопными отображениями.

Поясним требования к G выше. Для того, чтобы G гладко деформировалось в F необходимо найти непрерывно дифференцируемую гомотопию H , которая связывает эти функции. Классический пример — это гомотопия Ньютона:

$$H(x, \mu) = F(x) - (1 - \mu)F(x^0), \quad (1.2)$$

где $G(x) = F(x) - F(x^0)$ и $G(x^0) = 0$, $x^0 \in \Omega$.

Очевидно, что гладкость H полностью определяется F . В определении (1.1.1) вовсе не обязательно, чтобы $\mu \in [0, 1]$, всегда можно произвести такую гладкую замену параметра, чтобы μ принадлежало любому промежутку (даже интервалу, в том числе и бесконечному, в таком случае требования к H в определении (1.1.1) примут предельный вид).

Таким образом исходная задача (1.1) сводится к решению системы из n уравнений с $n + 1$ неизвестной

$$H(x, \mu) = 0, \quad (1.3)$$

причем $H(x^0, 0) = 0$. Нам необходимо найти все такие $x \in \Omega$, которые удовлетворяют уравнению $H(x, 1) = 0$. Дальнейшие рассуждения основываются на фундаментальной теореме из курса анализа о неявно заданном отображении.

Теорема 1.1.1. *(о неявно заданном отображении)*

Пусть отображение $H(x, y) : \mathbb{R}^n \times \mathbb{R}^m \supset U \rightarrow \mathbb{R}^n$ непрерывно дифференцируемо на открытом множестве U . Пусть в какой-то точке $A(x^0, y^0) \in U$ выполняется равенство $H(x^0, y^0) = 0$ и матрица частных производных по переменной y невырождена, т. е. $\det \frac{\partial H}{\partial y}(x^0, y^0) \neq 0$. Тогда:

1. существует окрестность $Q_A \subset U \subset \mathbb{R}^n \times \mathbb{R}^m$ точки $A(x^0, y^0)$, шаровая окрестность $X_{x^0} \subset \mathbb{R}^n$ точки x^0 и единственное неявное отображение $f(x) : X_{x^0} \rightarrow \mathbb{R}^m$, для которого условие разрешимости уравнения

$$H(x, y) = 0, \text{ где } (x, y) \in Q_A$$

эквивалентно условию

$$y = f(x), \text{ где } x \in X_{x^0},$$

2. отображение f непрерывно дифференцируемо на X_{x^0} и для любого $x \in X_{x^0}$ матрица Якоби вычисляется по формуле

$$J(f) \Big|_x = - \left(\frac{\partial H}{\partial y} \Big|_{(x, f(x))} \right)^{-1} \circ \frac{\partial H}{\partial x} \Big|_{(x, f(x))}. \quad (1.4)$$

Точки, в которых ранг матрицы Якоби максимален, будем называть *регулярными*, в противном случае — *особенными*.

Всегда можно добиться того, чтобы H было определено на некотором открытом множеством $O \supset \Omega \times [0, 1]$. Если выполняется неравенство $\det \frac{\partial H}{\partial x}(x^0, 0) \neq 0$, то все условия теоремы (1.1.1) соблюдены, и существует в некоторой ε -окрестности нуля единственная функция $f_0(\mu) : U(0, \varepsilon_0) \rightarrow \mathbb{R}^n$, график которой принадлежит множеству решений уравнения (1.3).

Доопределим $f_0(\mu)$ по непрерывности на концах интервала $U(0, \varepsilon_0)$. Заметим, что в силу гладкости $f_0(\mu)$, существуют односторонние производные, т. е. $f_0(\mu)$ непрерывно дифференцируема на замыкании $U(0, \varepsilon_0)$, причем граничные точки также являются решением (1.3).

Если $1 \leq \varepsilon_0$, то решение (1.1) найдено. Иначе возьмем точку $(x^1, \mu_1) = (f_0(\varepsilon_0), \varepsilon_0)$. При выполнении условия $\det \frac{\partial H}{\partial x}(x^1, \mu_1) \neq 0$ существует единственная функция $f_{\mu_1}(\mu) : U(\mu_1, \varepsilon_{\mu_1}) \rightarrow \mathbb{R}^n$, и ее график суть решение системы (1.3).

Можно и дальше продолжать продвигаться по параметру в предположении, что на каждом шаге $\det \frac{\partial H}{\partial x} \neq 0$. Тогда объединение всех графиков f_{μ_k} есть некоторая гладкая кривая Γ , которая может быть параметризована функцией $f(\mu) : I \rightarrow \mathbb{R}^n$, где $I = \bigcup_k \overline{U}(\mu_k, \varepsilon_{\mu_k})$ линейно связное множество в $\mathbb{R}$. Если замыкание I содержит $\mu = 1$, то замыкание $f(I)$ содержит решение уравнения (1.1).

Описанный подход не является конструктивным. Он лишь доказывает существование решения. Куда более перспективной выглядит дифференциальная форма задачи (1.3). Добавив к формуле (1.4) начальное условие $x(0) = x^0$ и сделав элементарные преобразования, получаем задачу Коши:

$$\frac{\partial H}{\partial x} \frac{dx}{d\mu} + \frac{\partial H}{\partial \mu} = 0, \quad x(0) = x^0, \quad (1.5)$$

где неизвестное $\frac{dx}{d\mu}$ входит линейно. Такой подход позволяет использовать различные конечно-разностные методы для поиска решения.

В рассуждениях выше предполагалось, что на каждом шаге матрица $\frac{\partial H}{\partial x}$ невырождена. В тех точках, где это условие нарушается — будем называть такие точки *предельными* относительно параметра —, возможны два варианта:

1. Если в точке (x, μ) определитель матрицы $\frac{\partial H}{\partial x}$ равен нулю, но при этом ранг матрицы Якоби максимален (т. е. (x, μ) — регулярная точка H), то продолжение решения возможно единственным образом с заменой параметра.
2. Если точка (x, μ) является особенной точкой отображения H , то продолжение решения в общем случае невозможно.

Продлить решение в особенной точке (даже с заменой параметра) далеко не всегда возможно, но иногда это удается сделать, пожертвовав единственностью продолжения. Возникает так называемое ветвление решения. Исчерпывающего описания подобного класса задач нет, но можно ознакомиться с достаточно подробной классификацией особенных точек в монографии [1], где также исследуется поведение решения в окрестностях особенных точек.

1.2 Замена параметра

Перепишем дифференциальное уравнение (1.5) следующим образом:

$$\sum_{j=1}^n \frac{\partial H}{\partial x_j} \frac{dx_j}{d\mu} + \frac{\partial H}{\partial \mu} = 0. \quad (1.6)$$

Введем обозначения

$$J = \begin{bmatrix} \frac{\partial H}{\partial x_1} & \frac{\partial H}{\partial x_2} & \cdots & \frac{\partial H}{\partial x_n} \end{bmatrix}, \quad \bar{J} = \begin{bmatrix} \frac{\partial H}{\partial x_1} & \frac{\partial H}{\partial x_2} & \cdots & \frac{\partial H}{\partial x_n} & \frac{\partial H}{\partial \mu} \end{bmatrix},$$

$$J_i = \begin{bmatrix} \frac{\partial H}{\partial x_1} & \frac{\partial H}{\partial x_2} & \cdots & \frac{\partial H}{\partial x_{i-1}} & \frac{\partial H}{\partial x_{i+1}} & \cdots & \frac{\partial H}{\partial x_n} & \frac{\partial H}{\partial \mu} \end{bmatrix}, \quad i = \overline{1, n}.$$

Тогда, если $\det J \neq 0$, то решение (1.6) по правилу Крамера можно представить как

$$\frac{dx_i}{d\mu} = (-1)^{n-i+1} \frac{\det J_i}{\det J}, \quad i = \overline{1, n}. \quad (1.7)$$

Предположим, что $\det J = 0$, но ранг $\bar{J}$ максимален ($\text{rank } \bar{J} = n$). Тогда среди определителей $\det J_i$, $i = \overline{1, n}$, хотя бы один не равен нулю. Пусть $\det J_k \neq 0$, в таком случае условия теоремы (1.1.1) в рассматриваемой точке выполняются, возможно продолжить решение уже по параметру x_k , и уравнение (1.6) принимает вид:

$$\sum_{j=1}^n \frac{\partial H}{\partial x_j} \frac{dx_j}{dx_k} + \frac{\partial H}{\partial \mu} \frac{d\mu}{dx_k} = 0. \quad (1.8)$$

Опять же по правилу Крамера имеем:

$$\frac{d\mu}{dx_k} = (-1)^{n-k+1} \frac{\det J}{\det J_k}, \quad \frac{dx_i}{dx_k} = (-1)^{k-i+1} \frac{\det J_i}{\det J_k}, \quad i \neq k. \quad (1.9)$$

При $\det J = 0$ вектор касательной к кривой Γ становится ортогонален оси μ . Действительно, как видно из (1.9), $\frac{d\mu}{dx_k} = 0$, и, следовательно, скалярное произведение касательного вектора $\vec{\tau}$ на орт $\vec{\mu}$ равно нулю:

$$\vec{\tau} = \left(\frac{dx_1}{dx_k} \quad \frac{dx_2}{dx_k} \cdots \frac{dx_n}{dx_k} \quad \frac{d\mu}{dx_k} \right)^T, \quad \vec{\mu} = (0 \quad 0 \cdots 0 \quad 1)^T,$$

$$\frac{d\mu}{dx_k} = 0 \Rightarrow (\vec{\tau}, \vec{\mu}) = 0.$$

Другими словами в окрестностях предельных точек решение (1.7) неограниченно растет, что гарантирует неустойчивость любой численной схемы. Положение спасает замена параметра, описанная выше, но ровно до того момента пока вектор $\vec{\tau}$ не станет ортогонален оси x_k .

Появляется справедливый вопрос: существует ли такая замена параметра, которая имела бы наилучшую устойчивость, скорость сходимости и погрешность для какой-нибудь численной схемы нахождения решения? Пока только можно сделать вывод, что нет принципиальной разницы с точки зрения продолжения решения между неизвестной x_k и параметром μ . Поэтому обозначим для краткости записи $\mu = x_{n+1}$, а $J = J_{n+1}$ здесь и далее.

1.3 Наилучшая параметризация

Озадачимся поиском наилучшего параметра. В качестве направления отсчета параметра можно взять одну из осей координат, так чтобы выполнялись требования теоремы (1.1.1). Недостатки такого подхода уже были рассмотрены. Одним из них является отсутствие универсальной параметризации. В общем случае для каждой точки кривой Γ «идеальное» направление параметра уникально. Важным требованием к этому «идеальному» направлению — оно не должно быть ортогонально вектору касательной.

Очевидным кандидатом является сама кривая Γ . Действительно, если отсчитывать параметр вдоль кривой Γ , то в любой точке направление параметра будет коллинеарно касательной. Оказывается (и это будет показано в дальнейшем), выбор такой параметризации гарантирует лучшую сходимость для произвольной численной схемы.

В качестве направления параметра s возьмем произвольный единичный вектор $\vec{\alpha} = (\alpha_1 \ \alpha_2 \ \dots \ \alpha_{n+1})^T$. Сам параметр s определим из соотношения:

$$ds = (\vec{\alpha}, dx), \quad dx = (dx_1 \ dx_2 \ \dots \ dx_{n+1})^T. \quad (1.10)$$

Если в качестве $\vec{\alpha}$ взять орт оси x_{n+1} , то $ds = dx_{n+1}$, и следовательно $s = \mu + C$. При $C = 0$ получаем исходную параметризацию.

Уравнение (1.5) с параметризацией s можно переписать так:

$$\bar{J} \frac{dx}{ds} = 0. \quad (1.11)$$

Система (1.11) с n уравнениями и с $n + 1$ неизвестной линейна относительно неизвестных $\frac{dx_i}{ds}$, $i = \overline{1, n+1}$. Замкнем систему, добавив условие (1.10):

$$\bar{J} \frac{dx}{ds} = 0, \quad \left(\vec{\alpha}, \frac{dx}{ds} \right) = 1, \quad (1.12)$$

или, что тоже самое, но в блочном виде

$$\begin{bmatrix} \alpha_1 & \alpha_2 & \dots & \alpha_{n+1} \\ \frac{\partial H}{\partial x_1} & \frac{\partial H}{\partial x_2} & \dots & \frac{\partial H}{\partial x_{n+1}} \end{bmatrix} \frac{dx}{ds} = (\delta_{1k})_{k=1}^{n+1}, \quad (1.13)$$

где δ_{ij} — символ Кронекера.

Необходимо научиться оценивать качество параметра s . Известно, что малость погрешности решения системы (1.13) обеспечивает хорошая обусловленность матрицы этой системы. Свяжем обусловленность матрицы с параметром s .

В [3] для оценки качества матрицы вводится понятие *числа обусловленности*, которое можно определить так

$$\mathbf{cond}(A) = \begin{cases} \|A\| \|A^{-1}\|, & \det(A) \neq 0, \\ +\infty, & \det(A) = 0, \end{cases} \quad (1.14)$$

где $\|\cdot\|$ — подчиненная норма матрицы. Причем из свойств подчиненных матричных норм следует

$$+\infty \geq \|A\| \|A^{-1}\| \geq \|AA^{-1}\| = \|E\| = 1.$$

Отсюда видно, что чем ближе число обусловленности к 1, тем лучше обусловлена матрица. В частности ортогональные матрицы являются «идеально» обусловленными, так как их подчиненная матричная норма всегда равна 1.

Число обусловленности для дальнейшего анализа затруднительно использовать напрямую. Введем другую меру обусловленности матрицы и свяжем ее с (1.14).

Определение 1.3.1. Пусть A — произвольная квадратная матрица порядка p . Мерой обусловленности матрицы A назовем число $\mathbf{meas}(A)$, которое определяется следующим образом:

1. Если $\det A = 0$, то $\mathbf{meas}(A) = 0$;
2. Если $\det A \neq 0$, то верна формула:

$$\mathbf{meas}(A) = \frac{\det A}{\prod_{i=1}^p |a_i|}, \quad |a_i| = \sqrt{(a_i, a_i)}, \quad (1.15)$$

где a_i — i -я строка матрицы A .

Как известно из курса анализа и линейной алгебры модуль определителя $|\det A|$ равен объему p -мерного параллелепипеда, построенного на векторах $\vec{a}_1, \vec{a}_2, \dots, \vec{a}_p$. Тогда формула (1.15) описывает ориентированный объем p -мерного параллелепипеда, построенного на единичных векторах $\vec{e}_1, \vec{e}_2, \dots, \vec{e}_p$, где $\vec{e}_i = \frac{\vec{a}_i}{|a_i|}$, $i = \overline{1, p}$. Очевидно, что если набор таких векторов линейно зависим, то объем равен нулю.

Теорема 1.3.1. Пусть A — произвольная квадратная матрица порядка p . Тогда справедливо двойное неравенство $0 \leq |\mathbf{meas}(A)| \leq 1$, причем нижняя грань неравенства достигается только в случае линейной зависимости строк матрицы A , а верхняя грань достигается тогда и только тогда, когда строки матрицы A попарно ортогональны.

Доказательство. Без ограничения общности будем считать, что строки матрицы A имеют единичную длину ($|a_i| = 1$, $i = \overline{1, p}$). Докажем утверждение по индукции.

1) При $p = 2$ (плоский случай):

Строки матрицы A образуют параллелограмм, площадь которого есть произведение основания на высоту:

$$S = |a_1||a_2|\sin(\varphi), \quad \varphi = \widehat{\vec{a}_1 \vec{a}_2},$$

а так как $|a_1| = |a_2| = 1$, то

$$|\mathbf{meas}(A)| = S = \sin(\varphi),$$

откуда следует утверждение теоремы.

2) Пусть утверждение теоремы верно для p . Докажем справедливость для $p + 1$.

В курсе анализа доказывается, что объем произвольного цилиндра есть произведение «площади» основания S на высоту h . Параллелепипед — частный случай цилиндра.

Основание суть p -мерный параллелепипед, построенный на векторах $\vec{a}_1, \vec{a}_2, \dots, \vec{a}_p$. Тогда для «площади» основания верно утверждение теоремы $0 \leq S \leq 1$, причем $S = 1$ только тогда, когда стороны параллелепипеда попарно ортогональны.

Возьмем произвольную единичную нормаль $\vec{n}$ гиперплоскости T_p , построенной на векторах $\vec{a}_1, \vec{a}_2, \dots, \vec{a}_p$. Тогда высота h определяется так:

$$h = |(\vec{a}_{p+1}, \vec{n})| = |a_{p+1}||\vec{n}|\cos(\theta) = |\cos(\theta)|, \quad \theta = \widehat{\vec{a}_{p+1} \vec{n}}.$$

Окончательно получаем

$$|\mathbf{meas}(A)| = S|\cos(\theta)| \leq 1,$$

где $|\mathbf{meas}(A)| = 1$ только тогда, когда $\vec{a}_{p+1}$ ортогонален гиперплоскости T_p и $S = 1$.

□

При решении системы линейных алгебраических уравнений, которая в матричном виде представима как $Ax = b$, можно поделить каждое уравнение на константу $|a_i|$ и получить систему $Qx = \hat{b}$. Если, дополнительно, предположить, что $|\mathbf{meas}(A)| = 1$, то Q — ортогональная матрица. Следовательно, чем ближе $|\mathbf{meas}(A)|$ к единице, тем лучше обусловлена матрица A .

Заметим, что класс «идеально» обусловленных матриц относительно $\mathbf{meas}(\cdot)$ шире, чем тот же класс относительно $\mathbf{cond}(\cdot)$. К первым относятся все невырожденные матрицы $A = DQ$, где D — диагональная, а Q — ортогональная матрицы. Ко вторым — только ортогональные матрицы.

Вернемся к системе уравнений (1.13). Найдём решение по правилу Крамера:

$$\frac{dx_i}{ds} = (-1)^{i+1} \frac{\det J_i}{\Delta}, \quad i = \overline{1, n+1}, \quad (1.16)$$

$$\Delta = \sum_{k=1}^{n+1} (-1)^{k+1} \alpha_k \det J_k. \quad (1.17)$$

Обозначим матрицу системы (1.13) как Φ . Исследуем свойства этой системы в фиксированной точке при переменном векторе $\vec{\alpha}$.

Теорема 1.3.2. *Мера обусловленности матрицы системы (1.13) достигает наибольшего по модулю значения в том случае, когда вектор $\vec{\alpha}$ коллинеарен вектору касательной кривой решения Γ в рассматриваемой точке.*

Доказательство. Исследуем на условный экстремум функционал $\mathbb{F}\vec{\alpha} = \mathbf{meas}(\Phi)$ при ограничении $(\vec{\alpha}, \vec{\alpha}) = 1$. Воспользуемся формулами (1.15) и (1.17), учитывая ограничение на $\vec{\alpha}$:

$$\mathbb{F}\vec{\alpha} = \frac{\Delta}{d}, \quad d = \prod_{i=2}^{n+1} |\varphi_i| = \mathbf{const}, \quad (1.18)$$

где φ_i , $i = \overline{1, n+1}$, — строки матрицы Φ .

Для решения задачи на условный экстремум составим функцию Лагранжа для функционала $\mathbb{F}$:

$$L = \mathbb{F}\vec{\alpha} + \lambda(1 - (\vec{\alpha}, \vec{\alpha})) = \sum_{i=1}^{n+1} \left((-1)^{i+1} \alpha_i \frac{\det J_i}{d} + \lambda(1 - \alpha_i \alpha_i) \right). \quad (1.19)$$

Тогда необходимое условие существования экстремума при заданных ограничениях выглядит следующим образом:

$$\frac{\partial L}{\partial \alpha_k} = (-1)^{k+1} \frac{\det J_k}{d} - 2\lambda \alpha_k = 0, \quad k = \overline{1, n+1}, \quad (1.20)$$

откуда можно легко выразить $\alpha_k = (-1)^{k+1} \frac{\det J_k}{2d\lambda}$. Подставив эти значения α_k в ограничение $(\vec{\alpha}, \vec{\alpha}) = 1$, получим выражение для λ :

$$\lambda = \pm \frac{\sqrt{\Delta_i \Delta_i}}{2d}, \quad \Delta_i \Delta_i = \sum_{i=1}^{n+1} \det J_i \det J_i. \quad (1.21)$$

Таким образом условный экстремум, если и достигается, то только при

$$\alpha_k = \pm (-1)^{k+1} \frac{\det J_k}{\sqrt{\Delta_i \Delta_i}}, \quad k = \overline{1, n+1}, \quad (1.22)$$

и накладывается ограничение на определитель матрицы Φ :

$$\Delta = \sum_{k=1}^{n+1} (-1)^{k+1} \left(\pm (-1)^{k+1} \frac{\det J_k}{\sqrt{\Delta_i \Delta_i}} \right) \det J_k = \pm \sqrt{\Delta_i \Delta_i}. \quad (1.23)$$

Но тогда из формул (1.16) и (1.22) следует, что

$$\alpha_k = (-1)^{k+1} \frac{\det J_k}{\Delta} = \frac{dx_k}{ds}, \quad k = \overline{1, n+1}. \quad (1.24)$$

Равенства (1.24) являются необходимыми для существования условного экстремума функционала $\mathbb{F}$. Покажем, что они являются и достаточными. Для этого рассмотрим квадратичную форму, порожденную вторым дифференциалом функции Лагранжа:

$$d^2 L(d\vec{\alpha}) = (-2\lambda E d\vec{\alpha}, d\vec{\alpha}) = -2\lambda (d\vec{\alpha}, d\vec{\alpha}), \quad (1.25)$$

где, исходя из (1.18), (1.21) и (1.23), следует, что $\lambda = \frac{\Delta}{2d} = \frac{\mathbf{meas}(\Phi)}{2}$.

Тогда при $\mathbf{meas}(\Phi) < 0$ достигается условный минимум $\mathbb{F}$, а при $\mathbf{meas}(\Phi) > 0$ — условный максимум. А так как строки φ_i , $i = \overline{2, n+1}$ принадлежат нормальному подпространству в рассматриваемой точке кривой Γ и образуют там базис, а вектор $\vec{\alpha}$ коллинеарен вектору касательной (в силу равенств (1.24)), то можно сделать вывод, что строки матрицы Φ линейно независимы, и, следовательно, $\mathbf{meas}(\Phi) \neq 0$. $\square$

Геометрическая интерпретация теоремы (1.3.2). Как было показано выше, $|\text{meas}(\Phi)|$ есть объем $n+1$ -мерного параллелепипеда, построенного на векторах $\vec{e}_i = \frac{\vec{\varphi}_i}{|\varphi_i|}$, $i = \overline{1, n+1}$. Причем $\vec{\varphi}_i$, $i = \overline{2, n+1}$, зависят только от точки $x \in \mathbb{R}^{n+1}$, но не от вектора $\vec{\alpha}$. Следовательно, если взять за основание n -мерный параллелепипед, построенный на векторах $\vec{e}_2, \vec{e}_3, \dots, \vec{e}_{n+1}$, то его «площадь» никак не зависит от вектора $\vec{\alpha}$ и, при фиксированной точке x , постоянна. Высота же есть норма проекции $\vec{\varphi}_1 = \vec{\alpha}$ на вектор касательной в точке x . Очевидно, что высота принимает наибольшее значение только тогда, когда вектор касательной и $\vec{\alpha}$ коллинеарны.

В численных алгоритмах решения системы (1.13) придется иметь дело с дискретной аппроксимацией матрицы Φ . Следовательно, целесообразно уточнить как ведет себя решение при возмущении матрицы и правой части системы (1.13) в зависимости от выбора вектора $\vec{\alpha}$.

Пусть $\frac{dy}{ds}$ — решение возмущенной системы $\hat{\Phi} \frac{dy}{ds} = \hat{b}$, а $\frac{dx}{ds}$ — точное решение системы (1.13). Квадратичная погрешность $\delta^2 = \left| \frac{dy}{ds} - \frac{dx}{ds} \right|^2$. Естественно, нам бы хотелось, чтобы δ^2 принимало наименьшее значение.

Теорема 1.3.3. *Квадратичная погрешность решения системы (1.13), возникающая при возмущении элементов матрицы и правой части этой системы, принимает наименьшее значение в том случае, когда вектор $\vec{\alpha}$ коллинеарен вектору касательной кривой Γ в рассматриваемой точке.*

Доказательство. «Очевидно» по трем причинам: во-первых, полное доказательство довольно громоздкое, во-вторых, оно повторяет идею доказательства теоремы (1.3.2), в-третьих, его можно найти в [1]. $\square$

Резюмируем: для лучшей устойчивости и погрешности решения системы (1.13), как следует из равенств (1.24), необходимо и достаточно, чтобы вектор $\vec{\alpha} = \frac{dx}{ds}$. Тогда наилучший параметр s определяется из соотношения (1.10):

$$\sum_{i=1}^{n+1} \left(\frac{dx_i}{ds} \right)^2 = 1, \quad (1.26)$$

$$ds = \pm \sqrt{\sum_{i=1}^{n+1} (dx_i)^2}, \quad (1.27)$$

где ds дифференциал ориентированной длины кривой Γ , а s — натуральный параметр.

2 Численные схемы

Как было показано, процессу продолжения решения системы нелинейных уравнений

$$H(x) = 0, \quad H : \mathbb{R}^{n+1} \rightarrow \mathbb{R}^n, \quad H(x^0) = 0 \quad (2.1)$$

по натуральному параметру s соответствует задача Коши

$$\begin{aligned} \frac{dx}{ds} &= \Phi^{-1} \vec{e}_{n+1}, \quad x(0) = x^0, \\ \Phi &= \begin{bmatrix} \bar{J}(x) \\ \frac{dx}{ds} \end{bmatrix}, \quad \vec{e}_{n+1} = \begin{pmatrix} 0 & 0 & \dots & 0 & 1 \end{pmatrix}^T, \end{aligned} \quad (2.2)$$

где $\bar{J}$ — матрица Якоби отображения H .

Система (2.2) нелинейна относительно производной $\frac{dx}{ds}$ (далее просто $\dot{x}$), поэтому для построения численной схемы можно аппроксимировать Φ некоторой матрицей $\hat{\Phi}(y, \vec{\alpha})$ с погрешностью ε в рассматриваемой точке:

$$\hat{\Phi}(y, \vec{\alpha}) = \begin{bmatrix} \bar{J}(y) \\ \vec{\alpha} \end{bmatrix}, \quad \|\hat{\Phi}(y, \vec{\alpha}) - \Phi\| < \varepsilon. \quad (2.3)$$

Единичный вектор $\vec{\alpha}$ должен с заданной точностью аппроксимировать $\dot{x}$ и при этом легко находиться.

При построении шагового процесса нахождения решения на k -ом шаге можно в качестве $\vec{\alpha}$ взять вектор $\dot{x}_{k-1}$, а так как мелкость шага можно сделать сколь угодно малой, то неравенство (2.3) всегда выполнимо. Остается вопрос: как быть при $k = 0$? В предположении, что H регулярна в точке x^0 , вектор $\dot{x}_{-1}$ можно взять как орт какой-то из осей координат, например, $\dot{x}_{-1} = \vec{e}_{n+1}$, если $\det J_{n+1}(x^0) \neq 0$ (далее без ограничения общности будем считать, что это требование всегда выполняется).

Проинтегрируем уравнение (2.2) на отрезке $[s, s + \Delta s]$:

$$x(s + \Delta s) - x(s) = \int_s^{s+\Delta s} \Phi^{-1} \vec{e}_{n+1} d\varsigma, \quad (2.4)$$

откуда легко выражается $x(s + \Delta s)$.

Различные конечно-разностные методы решения поставленной задачи получаются путем аппроксимации правой части (2.4) квадратурными формулами, разнообразие которых подробно отражено в [3].

2.1 Конечно-разностные схемы

2.1.1 Простой метод Эйлера

В основе метода лежит квадратурная формула левых прямоугольников.

Инициализация

$$x_0 = x^0, \quad \dot{x}_{-1} = \vec{e}_{n+1}.$$

Переход к следующему шагу

$$\dot{x}_k^* = \hat{\Phi}^{-1}(x_k, \dot{x}_{k-1})\vec{e}_{n+1}, \quad \dot{x}_k = \frac{\dot{x}_k^*}{|\dot{x}_k^*|},$$

$$x_{k+1} = x_k + \dot{x}_k \Delta s_k, \quad s_{k+1} = s_k + \Delta s_k.$$

На k -ом шаге Φ аппроксимируется матрицей $\hat{\Phi}(x_k, \dot{x}_{k-1})$, при этом для нахождения вектора $\dot{x}_k^*$ необязательно вычислять обратную матрицу — эта символическая запись означает результат решения системы линейных уравнений любым известным способом.

В силу того, что $\bar{J}(x_k)\dot{x}_k^* = 0$, вектор $\dot{x}_k^*$ всегда является касательным в точке x_k . Уравнение $(\dot{x}_k^*, \dot{x}_{k-1}) = 1$ задает длину и одно из двух возможных направлений вектора $\dot{x}_k^*$. Действительно, скалярное произведение двух векторов положительно только тогда, когда угол между ними острый.

Направление обхода кривой решения Γ задается шагом и начальным приближением $\dot{x}_{-1}$. Для того чтобы на каком-то шаге ориентация кривой резко не изменилась, между производной и ее аппроксимацией должен быть острый угол, что вполне выполнимо при достаточной малости шага.

Отметим также, что на каждом шаге аппроксимация производной должна быть единичным вектором, для чего производится нормировка.

2.1.2 Модифицированный метод Эйлера

В основе метода лежит квадратурная формула трапеций.

Инициализация

$$x_0 = x^0, \quad \dot{x}_{-1} = \vec{e}_{n+1}.$$

Переход к следующему шагу

1. *Прогноз:*

$$\dot{x}_k^{p*} = \hat{\Phi}^{-1}(x_k, \dot{x}_{k-1})\vec{e}_{n+1}, \quad \dot{x}_k^p = \frac{\dot{x}_k^{p*}}{|\dot{x}_k^{p*}|},$$

$$x_{k+1}^p = x_k + \dot{x}_k^p \Delta s_k.$$

2. *Коррекция:*

$$\dot{x}_k^{c*} = \hat{\Phi}^{-1}(x_{k+1}^p, \dot{x}_k^p)\vec{e}_{n+1}, \quad \dot{x}_k^c = \frac{\dot{x}_k^{c*}}{|\dot{x}_k^{c*}|},$$

$$x_{k+1} = x_k + \frac{\dot{x}_k^p + \dot{x}_k^c}{2} \Delta s_k,$$

$$\dot{x}_k = \dot{x}_k^p, \quad s_{k+1} = s_k + \Delta s_k.$$

2.1.3 Метод Рунге-Кутты четвертого порядка точности

В основе этого метода лежит квадратурная формула Симпсона.

Инициализация

$$x_0 = x^0, \quad \dot{x}_{-1} = \vec{e}_{n+1}.$$

Переход к следующему шагу

1. *Прогноз:*

$$\dot{x}_k^{p*} = \hat{\Phi}^{-1}(x_k, \dot{x}_{k-1})\vec{e}_{n+1}, \quad \dot{x}_k^p = \frac{\dot{x}_k^{p*}}{|\dot{x}_k^{p*}|},$$

$$x_{k+\frac{1}{2}}^p = x_k + \dot{x}_k^p \frac{\Delta s_k}{2},$$

$$\dot{x}_{k+\frac{1}{2}}^{p*} = \hat{\Phi}^{-1}(x_{k+\frac{1}{2}}^p, \dot{x}_k^p)\vec{e}_{n+1}, \quad \dot{x}_{k+\frac{1}{2}}^p = \frac{\dot{x}_{k+\frac{1}{2}}^{p*}}{|\dot{x}_{k+\frac{1}{2}}^{p*}|},$$

$$x_{k+\frac{1}{2}}^c = x_k + \dot{x}_{k+\frac{1}{2}}^p \frac{\Delta s_k}{2},$$

$$\dot{x}_{k+\frac{1}{2}}^{c*} = \hat{\Phi}^{-1}(x_{k+\frac{1}{2}}^c, \dot{x}_{k+\frac{1}{2}}^p)\vec{e}_{n+1}, \quad \dot{x}_{k+\frac{1}{2}}^c = \frac{\dot{x}_{k+\frac{1}{2}}^{c*}}{|\dot{x}_{k+\frac{1}{2}}^{c*}|},$$

$$x_{k+1}^p = x_k + \dot{x}_{k+\frac{1}{2}}^c \Delta s_k,$$

2. *Коррекция:*

$$\dot{x}_{k+1}^{c*} = \hat{\Phi}^{-1}(x_{k+1}^p, \dot{x}_{k+\frac{1}{2}}^c)\vec{e}_{n+1}, \quad \dot{x}_{k+1}^c = \frac{\dot{x}_{k+1}^{c*}}{|\dot{x}_{k+1}^{c*}|},$$

$$x_{k+1} = x_k + \frac{\dot{x}_k^p + 2\dot{x}_{k+\frac{1}{2}}^p + 2\dot{x}_{k+\frac{1}{2}}^c + \dot{x}_{k+1}^c}{6} \Delta s_k,$$

$$\dot{x}_k^* = \dot{x}_k^p + 2\dot{x}_{k+\frac{1}{2}}^p + 2\dot{x}_{k+\frac{1}{2}}^c + \dot{x}_{k+1}^c, \quad \dot{x}_k = \frac{\dot{x}_k^*}{|\dot{x}_k^*|},$$

$$s_{k+1} = s_k + \Delta s_k.$$

2.2 Возможные эвристики

2.2.1 Сохранение ориентации кривой

Как уже было замечено, при ограничении $|\dot{y}| = 1$ уравнение $\bar{J}(x)\dot{y} = 0$ имеет два решения: $\dot{x}$ и $-\dot{x}$. Добавив условие $(\vec{\alpha}, \dot{y}) = 1$, мы однозначно определяем знак производной, а следовательно, и ориентацию кривой в окрестности точки x .

Пусть вектор-функция $\vec{\tau}(s)$ является натуральной параметризацией ориентированной кривой Γ . Тогда решением системы (2.2) будет вектор $\dot{x} = \frac{d\vec{\tau}}{ds}$. Если угол между векторами $\frac{d\vec{\tau}}{ds}$ и $\vec{\alpha}$ острый (т.е. $(\frac{d\vec{\tau}}{ds}, \vec{\alpha}) > 0$), то $\dot{y} = \dot{x}$. В противном случае, либо $\dot{y}$ не существует (когда $(\frac{d\vec{\tau}}{ds}, \vec{\alpha}) = 0$), либо $\dot{y} = -\dot{x}$ (когда $(\frac{d\vec{\tau}}{ds}, \vec{\alpha}) < 0$). Возникает вопрос о сохранении ориентации кривой Γ при решении возмущенной системы (2.2).

Как известно из курса линейной алгебры множество базисов в линейном пространстве разбивается на два непересекающихся класса: *правый* и *левый*. Определить эти классы можно следующим образом. Естественнo положить канонический базис правым. Тогда строки любой невырожденной матрицы A при условии $\det A > 0$ образуют правый базис. Если же $\det A < 0$, то — левый базис.

В силу регулярности точки x матрица Φ (с непрерывными элементами по x) невырождена, следовательно $\det \Phi$ знакопостоянная функция. Потребуем, чтобы строки Φ образовывали правый базис в точке x . Тогда, для сохранения ориентации вектора $\dot{y}$, необходимо и достаточно, чтобы строки матрицы $\hat{\Phi}(x, \dot{y})$ тоже образовывали правый базис.

Продemonстрируем указанный подход на примере. Рассмотрим произвольный шаговый процесс, описанный выше. Как легко заметить, резкое изменение ориентации кривой влечет за собой численную неустойчивость. Следовательно, полезно научиться «выправлять» неправильное направление вектора касательной.

Пусть задана правая ориентация кривой. На каждом новом шаге вычисляется

$$\dot{y}_k^* = \hat{\Phi}^{-1}(x_k, \dot{x}_{k-1})\vec{e}_{n+1}, \quad \dot{y}_k = \frac{\dot{y}_k^*}{|\dot{y}_k^*|},$$

где $\dot{x}_{k-1}$ выступает в роли $\vec{\alpha}$ для точки x_k . Тогда

$$\dot{x}_k = \mathbf{sign}(\det \hat{\Phi}(x_k, \dot{y}_k)) \cdot \dot{y}_k, \quad \mathbf{sign}(x) = \frac{x}{|x|}, \quad x \neq 0.$$

Таким образом, мы на каждом шаге сохраняем ориентацию кривой Γ . Аналогичную процедуру можно проделать и в случае с левой ориентацией.

2.2.2 Динамический размер шага

Во всех описанных численных схемах для достижения наилучшей погрешности требуется уменьшить шаг интегрирования h , что влечет увеличение времени выполнения программы. Маленький шаг на всем отрезке интегрирования является не оптимальным решением. Куда продуктивнее динамически изменять шаг в зависимости от локальной специфики правой части (2.2). Предлагается следующий подход:

1. Вводится специальный параметр $err \geq 0$. Его можно определить на k -ом шаге, например, как $|H(x_{k+1})|$ или $|\dot{x}_{k-1} - \dot{x}_k|$.
2. Если $err \leq \delta$, т. е. ошибка меньше или равна желаемой погрешности, то полученное решение x_{k+1} считается допустимым и интегрирование продолжается с новым шагом $h = \min(h_{max}, \gamma h)$, где $\gamma \geq 1$, а h_{max} — ограничение на максимальный шаг, вводится пользователем.
3. Если же $err > \delta$, то решение на k -ом шаге пересчитывается с шагом $h = \max(h_{min}, \gamma h)$, где $0 < \gamma < 1$, а h_{min} — ограничение на минимальный шаг, вводится пользователем.

Функцию γ предлагается ввести следующим образом:

$$\gamma(err) = \min \left(f_{max}, \max \left(f_{min}, \left(\frac{\delta}{err} \right)^{\frac{1}{m+1}} \right) \right),$$

где $f_{max} \geq 1$ ($0 < f_{min} < 1$) — максимальный (минимальный) коэффициент увеличения шага, а m — порядок точности численной схемы.

Такой подход обеспечивает изменение шага h в зависимости от скорости изменения искомого решения. Там, где решение меняется плавно, программа автоматически переходит на более крупный шаг и, наоборот, при более быстром изменении решения шаг уменьшается. Это позволяет существенно ускорить решение задачи, не снижая точности.

Заключение

Список литературы

- [1] В. И. Шалапилин, Е. Б. Кузнецов, *Метод продолжения решения по параметру и наилучшая параметризация (в прикладной математике и механике)*. — М.: Эдиториал УРСС, 1999. — 224 с.
- [2] Красносельский А.М., Вайникко Г.М., Забрейко П.П. и др., *Приближенное решение операторных уравнений*. — М.: Наука, 1969. — 457 с.
- [3] Бахвалов Н. С. , *Численные методы*. — М.: Наука, 1973. — 636 с.
- [4] Васильев В. А., *Введение в топологию*. — М.: ФАЗИС, 1997. — 132 с.
- [5] Давиденко Д. Ф., *О приближенном решении систем нелинейных уравнений*.
// Украинский матем. ж. 1953. Т. 5. № 2. С. 196-206.
- [6] Lahaye M. E., *Une methode de resolution d'une categorie d'equations transcendentes*. // Compter Rendus heb domataires des seances de L'Acad. sci. 1934. V. 198. № 21. P. 1840-1842.

Приложение А Программная реализация

Подключение зависимостей и объявление базовых классов **BaseSolver** и **BaseEquation**.

```
1  import warnings
2  from abc import ABC, abstractmethod, ABCMeta
3  import numpy as np
4  from numpy.linalg import solve as lin_solve, vector_norm, matrix_rank,
   ↪ LinAlgError, det
5  import matplotlib.pyplot as plt
6  from enum import IntEnum
7
8
9  class DecisionNotContinueWarning(UserWarning):
10     """Предупреждение о невозможности продолжения решения"""
11
12
13  class OrientationDecision(IntEnum):
14     RIGHT = 1
15     LEFT = -1
16
17
18  class BaseSolver(ABC):
19
20     def __init__(self, h_min: float, h_max: float, eps: float, orientation:
   ↪ OrientationDecision):
21         assert abs(h_min) <= abs(h_max)
22         assert 1e-16 <= h_min <= 1e-1
23         assert 1e-16 <= h_max <= 1e-1
24         assert 1e-32 <= eps <= 1e-1
25         self._h_min = h_min
26         self._h_max = h_max
27         self._eps = eps
28         self._orientation = orientation
29
30     def change_orientation(self, new_orientation: OrientationDecision):
31         self._orientation = new_orientation
32
```

```
33     def _set_correction(self, fac_min, fac_max, m):
34         tol = self._eps
35
36         def wrapped(err):
37             if err == 0:
38                 return fac_max
39             return min((fac_max, max(fac_min, (tol / err) ** (1 / (m + 1)))))
40
41         return wrapped
42
43     def _new_tau(self, eq):
44         n = eq.n
45         diff = eq.differentiate
46         _norm = vector_norm
47         _solve = lin_solve
48         b = np.zeros(n, dtype="float64")
49         b[n - 1] = 1
50         orientation = self._orientation
51         sign = np.sign
52
53         def wrapped(x, tau):
54             new_tau = _solve(np.vstack((diff(x), tau)), b)
55             new_tau /= _norm(new_tau)
56             sgn = orientation * sign(det(np.vstack((diff(x), new_tau))))
57
58             return sgn * new_tau
59
60         return wrapped
61
62     @staticmethod
63     def _assert_start_approx(equation, x_0, eps):
64         assert vector_norm(equation.vector_func(x_0)) <= eps, f"Точка {x_0} не
        ↳ является решением уравнения."
65         assert matrix_rank(equation.differentiate(x_0)) == equation.n - 1,
        ↳ f"Точка {x_0} не является регулярной."
66
67
68
```



```
69     @staticmethod
70     def _find_start_vector(equation, x_0):
71         n = equation.n
72         tau = np.zeros(n, dtype="float64")
73         tau[n - 1] = 1
74         jacobian_matrix = equation.differentiate(x_0)
75         if not matrix_rank(np.vstack((jacobian_matrix, tau))) == n:
76             tau[n - 1] = 0
77             tau[n - 2] = 1
78         return tau
79
80     @abstractmethod
81     def __call__(self, equation, x_0, s_max):
82         raise NotImplementedError()
83
84
85 class BaseEquation(ABC):
86     @property
87     @abstractmethod
88     def n(self):
89         raise NotImplementedError()
90
91     @abstractmethod
92     def differentiate(self, x):
93         raise NotImplementedError()
94
95     @abstractmethod
96     def vector_func(self, x):
97         raise NotImplementedError()
98
99     def solve(self, solver: BaseSolver, x_0, s_max):
100         return solver(self, x_0, s_max)
```

а) Простой метод Эйлера:

```
1 class SimpleEulerSolver(BaseSolver):
2     _MAX_ITERS = 10
3
4     def __call__(self, eq: BaseEquation, x_0, s_max):
5         h_min = self._h_min
6         h_max = self._h_max
7         h = (h_max + h_min) / 2
8         eps = self._eps
9         correcting = self._set_correction(0.5, 2, 1)
10        new_tau = self._new_tau(eq)
11        func = eq.vector_func
12        _norm = vector_norm
13        BaseSolver._assert_start_approx(eq, x_0, eps)
14        tau = BaseSolver._find_start_vector(eq, x_0)
15        max_iters = self._MAX_ITERS
16        x = x_0
17        s = 0
18        cords = []
19        new_h = h
20        while abs(s) < s_max:
21            try:
22                tau_p = new_tau(x, tau)
23                cords.append((x, tau_p))
24                x_p = x + tau_p * h
25
26            for _ in range(max_iters):
27                err = _norm(func(x_p))
28                gamma = correcting(err)
29                if err < eps:
30                    new_h = min(h * gamma, h_max)
31                    break
32                else:
33                    new_h = max(h * gamma, h_min)
34                h = new_h
35                x_p = x + tau_p * h
36
```

```
37         tau = tau_p
38         x = x_p
39     except (LinAlgError, ZeroDivisionError):
40         warnings.warn("Дальнейшее продолжение решения невозможно.",
41                       ↪ DecisionNotContinueWarning)
42         break
43     s += h
44     h = new_h
45     return cords
```

b) Модифицированный метод Эйлера:

```
1 class ModifiedEulerSolver(BaseSolver):
2     _MAX_ITERS = 10
3
4     def __call__(self, eq: BaseEquation, x_0, s_max):
5         h_min = self._h_min
6         h_max = self._h_max
7         h = (h_max + h_min) / 2
8         eps = self._eps
9         correcting = self._set_correction(0.5, 2, 2)
10        new_tau = self._new_tau(eq)
11        _norm = vector_norm
12        BaseSolver._assert_start_approx(eq, x_0, eps)
13        tau = BaseSolver._find_start_vector(eq, x_0)
14        max_iters = self._MAX_ITERS
15        x = x_0
16        s = 0
17        cords = []
18        new_h = h
19        while abs(s) < s_max:
20            try:
21                tau_p = new_tau(x, tau)
22                cords.append((x, tau_p))
23                x_p = x + tau_p * h
24                tau_c = new_tau(x_p, tau_p)
25
```

```

26         for _ in range(max_iters):
27             err = _norm(tau_p - tau_c)
28             gamma = correcting(err)
29             if err < eps:
30                 new_h = min(h * gamma, h_max)
31                 break
32             else:
33                 new_h = max(h * gamma, h_min)
34
35             h = new_h
36             x_p = x + tau_p * h
37             tau_c = new_tau(x_p, tau_p)
38
39             x = x + (tau_p + tau_c) / 2 * h
40             tau = tau_c
41
42         except (LinAlgError, ZeroDivisionError):
43             warnings.warn("Дальнейшее продолжение решения невозможно.",
44                             ↪ DecisionNotContinueWarning)
45             break
46         s += h
47         h = new_h
48     return cords

```

с) Метод Рунге-Кутты четвертого порядка точности:

```

1 class RungeKuttSolver(BaseSolver):
2     _MAX_ITERS = 10
3
4     def __call__(self, eq: BaseEquation, x_0, s_max):
5         h_min = self._h_min
6         h_max = self._h_max
7         h = (h_max + h_min) / 2
8         eps = self._eps
9         correcting = self._set_correction(0.5, 2, 4)
10        new_tau = self._new_tau(eq)
11        _norm = vector_norm

```

```
12     BaseSolver._assert_start_approx(eq, x_0, eps)
13     tau = BaseSolver._find_start_vector(eq, x_0)
14     max_iters = self._MAX_ITERS
15     x = x_0
16     s = 0
17     cords = []
18     new_h = h
19     while abs(s) < s_max:
20         try:
21             k1 = new_tau(x, tau)
22             cords.append((x, k1))
23             x_1 = x + h / 2 * k1
24             k2 = new_tau(x_1, k1)
25             x_2 = x + h / 2 * k2
26             k3 = new_tau(x_2, k2)
27             x_3 = x + h * k3
28             k4 = new_tau(x_3, k3)
29             for _ in range(max_iters):
30
31                 err = _norm(k1 - k4)
32                 gamma = correcting(err)
33                 if err < eps:
34                     new_h = min(h * gamma, h_max)
35                     break
36                 else:
37                     new_h = max(h * gamma, h_min)
38
39             h = new_h
40
41             x_1 = x + h / 2 * k1
42             k2 = new_tau(x_1, k1)
43             x_2 = x + h / 2 * k2
44             k3 = new_tau(x_2, k2)
45             x_3 = x + h * k3
46             k4 = new_tau(x_3, k3)
47
48             x = x + h * (k1 + 2 * k2 + 2 * k3 + k4) / 6
49             tau = k4
```

```

50
51         except (LinAlgError, ZeroDivisionError):
52             warnings.warn("Дальнейшее продолжение решения невозможно.",
53                             ↳ DecisionNotContinueWarning)
54             break
55             s += h
56             h = new_h
57
58     return cords

```

Пример использования ($H(x, y) = \sin 40x - y = 0$):

```

1  class SomeEquation(BaseEquation):
2      n = 2  # Указывается число неизвестных в уравнении
3
4      def vector_func(self, x):  # Вычисляет значение отображения в точке x
5          return np.array([np.sin(40 * x[0]) - x[1]], dtype="float64")
6
7      def differentiate(self, x):  # Вычисляет матрицу Якоби в точке x
8          return np.array([40 * np.cos(40 * x[0]), -1], dtype="float64")
9
10
11  if __name__ == '__main__':
12      # h_min - минимальный размер шага
13      # h_max - максимальный размер шага
14      # eps - допустимая погрешность
15      # orientation - ориентация кривой
16      euler_solver = ModifiedEulerSolver(h_min=0.0001, h_max=0.005, eps=1e-4,
17                                          ↳ orientation=OrientationDecision.RIGHT)
18      eq = SomeEquation()
19      # x_0 - начальное условие задачи Коши
20      # s_max - максимальная длина кривой
21      res = eq.solve(solver=euler_solver, x_0=np.array([0, 0], dtype="float64"),
22                    ↳ s_max=10)

```

Приложение В Примеры и тесты

Амперсанд (кривая). Кривая задается уравнением

$$H(x, y) = (y^2 - x^2)(x - 1)(2x - 3) - 4(x^2 + y^2 - 2x)^2 = 0.$$

Точки $(0, \pm \frac{\sqrt{3}}{2})$, $(0, 0)$ и $(1, \pm 1)$ принадлежат множеству решений этого уравнения. Причем точки $(0, 0)$ и $(1, \pm 1)$ являются особенными, а $(0, \pm \frac{\sqrt{3}}{2})$ — регулярными. Тогда в качестве начального приближения можно взять $(0, \pm \frac{\sqrt{3}}{2})$. Ниже представлен график, демонстрирующий решение данного уравнения тремя численными схемами. Шаг интегрирования $10^{-3} \leq h \leq 10^{-2}$.

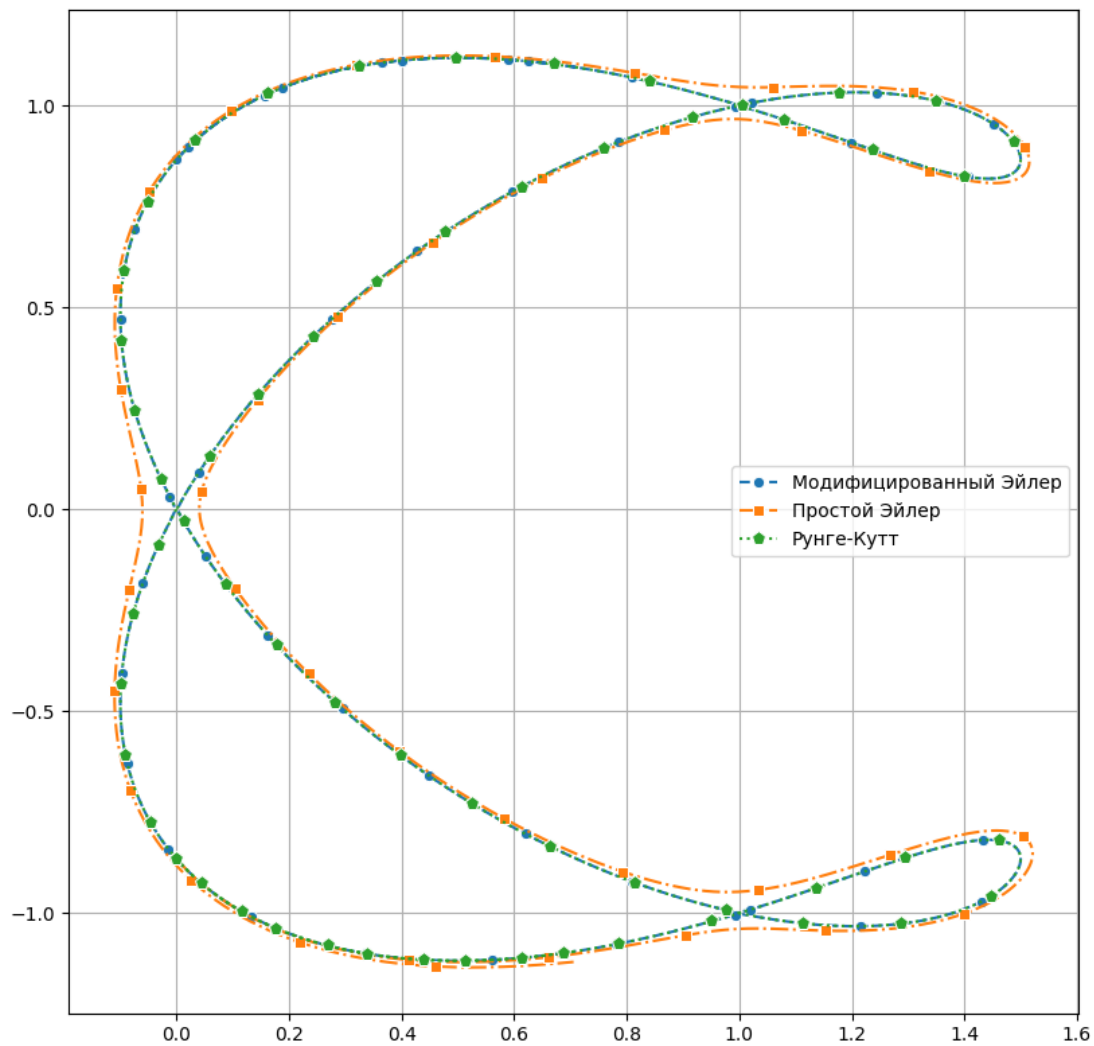


Рис. 1: Амперсанд.

Простой метод Эйлера сильнее всего расходится в окрестностях особых точек. В то время как модифицированный метод Эйлера и метод Рунге-Кутты устойчивы на протяжении всего обхода кривой решения (ошибка накапливается очень медленно).

Погрешность ψ вычисляется как $\max_k |H(x_k)|$, где x_k - решение, полученное той или иной численной схемой:

- Простой метод Эйлера — $\psi = 1.5 \times 10^{-2}$;
- Модифицированный метод Эйлера — $\psi = 6.224 \times 10^{-6}$;
- Метод Рунге-Кутты — $\psi = 1.324 \times 10^{-6}$.

Двукаспидная кривая. Уравнение кривой:

$$H(x, y) = (x^2 - 1)(x - 1)^2 + (y^2 - 1)^2 = 0,$$

где точки $(1, \pm 1)$ — особенные, а $(0, 0)$ — регулярная.

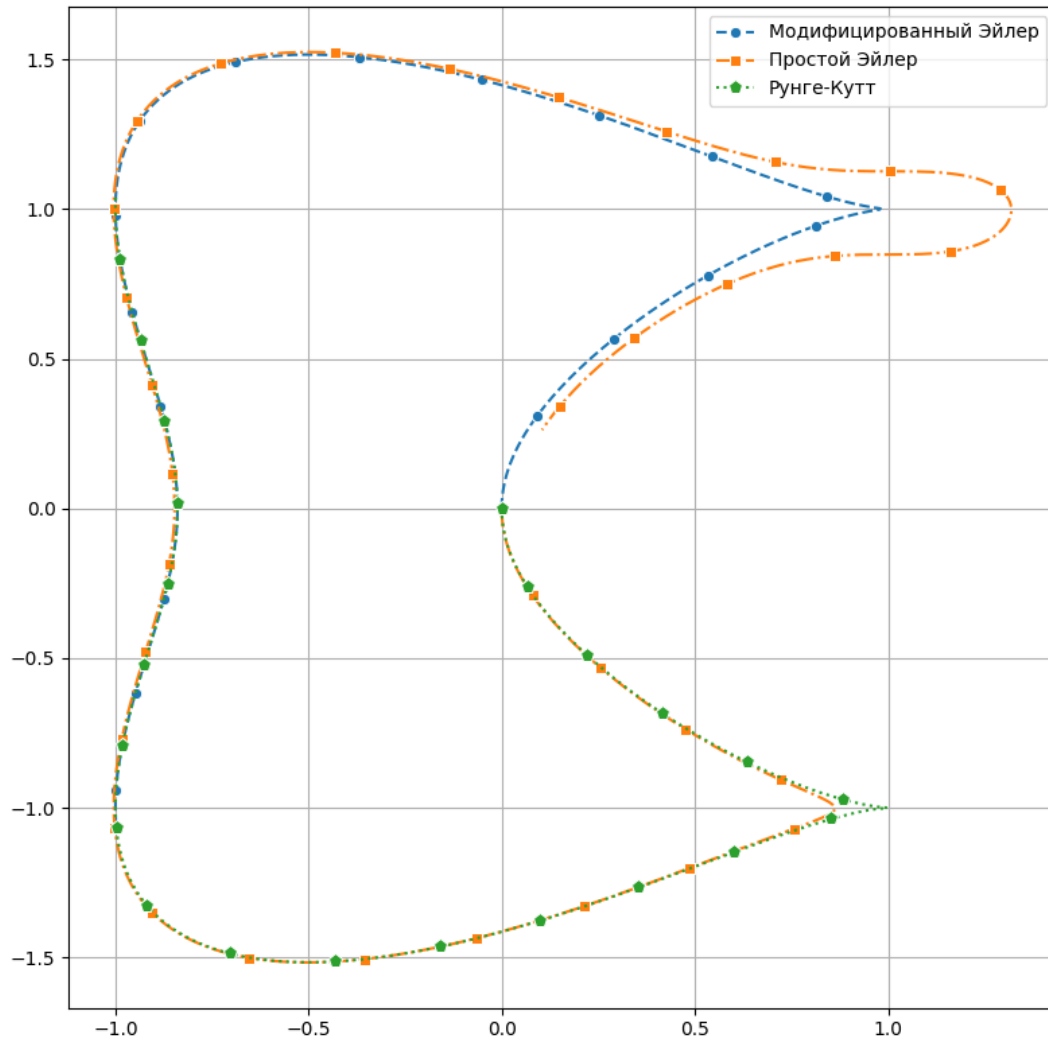


Рис. 2: Двукаспидная кривая.

Шаг интегрирования варьируется от 10^{-3} до 10^{-2} . Погрешности:

- Простой метод Эйлера — $\psi = 1.4 \times 10^{-2}$;
- Модифицированный метод Эйлера — $\psi = 2.022 \times 10^{-6}$;
- Метод Рунге-Кутты — $\psi = 3.692 \times 10^{-9}$.

Метод Рунге-Кутты показал себя наилучшим образом.

Трёхлистный клевер. Уравнение кривой:

$$H(x, y) = x^4 + 2x^2y^2 + y^4 - x^3 + 3xy^2 = 0,$$

где точки $(0, 0)$ — особенная точка, а $(1, 0)$ — регулярная.

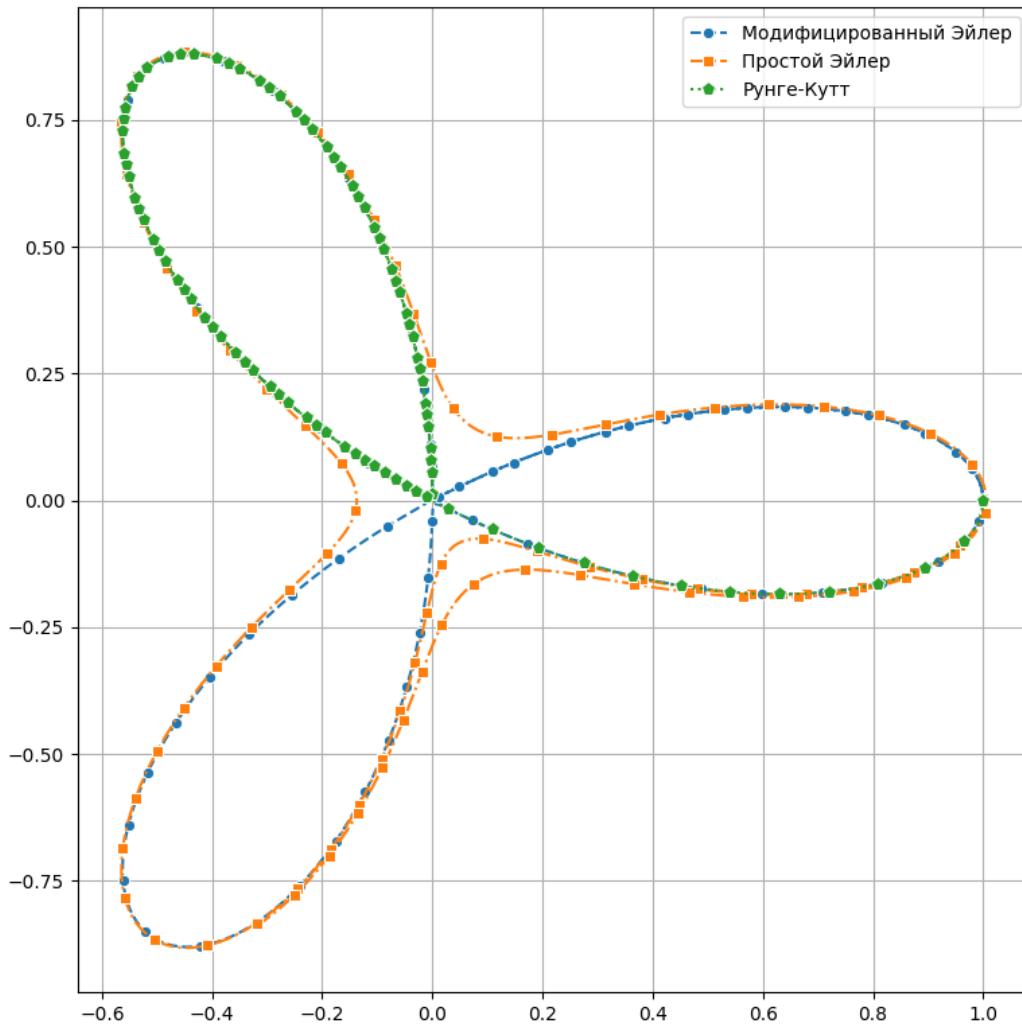


Рис. 3: Трёхлистный клевер.

Погрешности ($10^{-3} \leq h \leq 10^{-2}$):

- Простой метод Эйлера — $\psi = 6 \times 10^{-3}$;
- Модифицированный метод Эйлера — $\psi = 7.685 \times 10^{-7}$;
- Метод Рунге-Кутты — $\psi = 2.548 \times 10^{-10}$.

Трансцендентное уравнение.

$$H(x, y) = \frac{\sin x \cos y}{\sin(12xy) + 2y^2 + 2} - y^2 = 0.$$

Точки $(\pi k, 0)$, где $k \in \mathbb{Z}$, принадлежат множеству решений уравнения и отлично подходят для начального условия задачи Коши.

При $k = 0$:

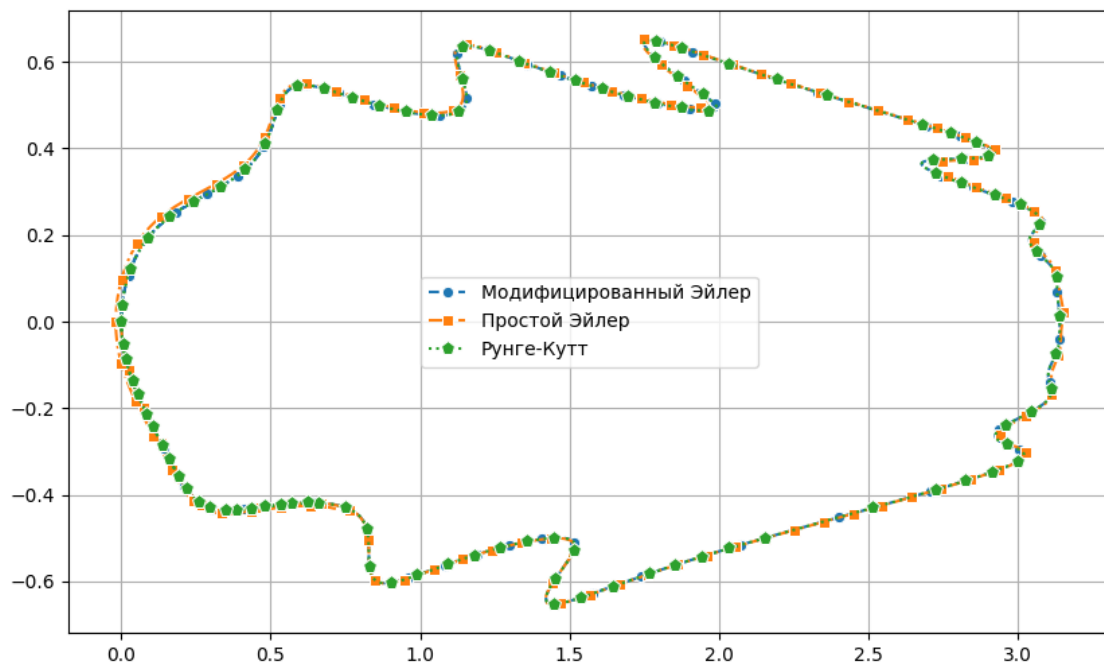
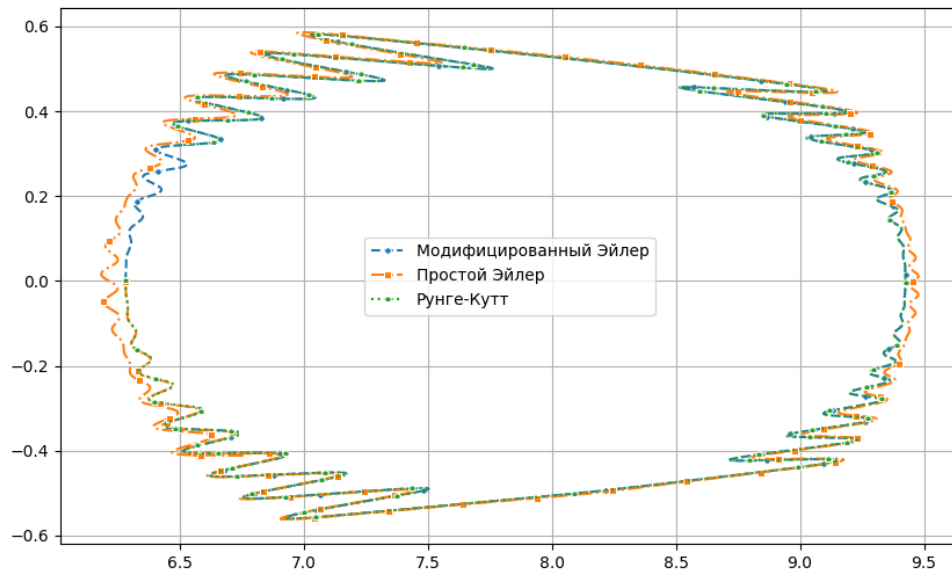
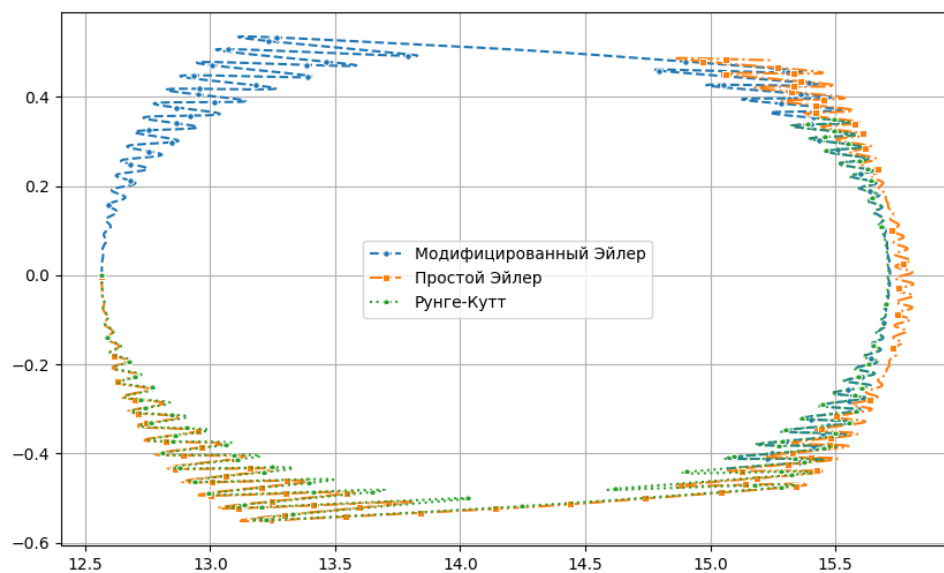


Рис. 4: Трансцендентное уравнение ($k = 0$).

Погрешности ($10^{-3} \leq h \leq 10^{-2}$):

- Простой метод Эйлера — $\psi = 10^{-2}$;
- Модифицированный метод Эйлера — $\psi = 1.064 \times 10^{-5}$;
- Метод Рунге-Кутты — $\psi = 2.79 \times 10^{-8}$.

(a) Решение при начальном приближении $(2\pi, 0)$.(b) Решение при начальном приближении $(4\pi, 0)$.Рис. 5: Трансцендентное уравнение ((a) при $k = 2$ и (b) при $k = 4$).

Погрешности при $k = 2$ ($k = 4$):

- Простой метод Эйлера — $\psi = 3 \times 10^{-1}$ ($\psi = 4 \times 10^{-1}$);
- Модифицированный метод Эйлера — $\psi = 9 \times 10^{-4}$ ($\psi = 5 \times 10^{-3}$);
- Метод Рунге-Кутты — $\psi = 8.12 \times 10^{-5}$ ($\psi = 1.8 \times 10^{-3}$).